Fail-Closed VPN Gateway on Linux

Policy Routing, nftables, and Transactional Updates — a design deep dive

Vassilis Dionysopoulos · July 2026 · Published at cloudtales.gr · Source (MIT): github.com/vdionisopoulos/nordvpn-linux-gateway-panel

A fail-closed VPN gateway routes selected devices through a VPN tunnel and guarantees one thing above all else. If the tunnel drops, those devices lose internet access instead of leaking to the ISP. I built one because some devices simply cannot protect themselves. Smart TVs, consoles, and streaming boxes have no VPN client and no way to get one. The usual fix is "put the VPN on the router." On a typical consumer router that is an all-or-nothing decision, and it drags every device in the house through the tunnel.

I wanted something more surgical: a small Ubuntu gateway that tunnels only the devices I choose, fails closed by design, and works from a phone browser. The interesting part is not the VPN. The same principles I apply to regulated cloud workloads apply here too: fail-closed defaults, least privilege, observable state, and transactional change management.

The problem, precisely stated

I set these requirements before writing any code:

- Route specific LAN devices, selected by IPv4 address, through NordVPN NordLynx, NordVPN's WireGuard-based tunnel technology. Leave every other device untouched.
- **Fail closed.** When the tunnel is down, managed devices must lose internet access entirely. Silent fallback to the ISP route is the one unacceptable outcome. For a privacy gateway, a leak is worse than an outage.
- Close the DNS side channel. Tunneling the traffic while DNS queries go to the ISP resolver is a well-known half-measure.
- Provide a LAN-only web panel for adding devices and switching exit country. The whole thing must work without SSH.
- Make updates transactional. They either fully succeed or fully roll back. A half-applied update on a security gateway is the worst possible state.

These requirements produce a classic control plane / data plane split. The data plane is a root systemd service written in Bash. It reconciles kernel routing and nftables state against a JSON configuration file. The control plane is a small Flask application running as an unprivileged user. It only edits that JSON file and talks to the NordVPN daemon. The two never share privileges. They share a file.

Why a fail-closed VPN gateway needs a blackhole route

Per-device routing on Linux is a policy routing problem. Each managed device gets an `ip rule` that sends its traffic to a dedicated routing table:

```
ip -4 rule add priority 10000 from 192.168.1.50/32 table 200
```

The fail-closed guarantee lives inside table 200 itself. The table always contains a low-priority blackhole default route. The VPN default route exists only while the tunnel interface is up and holds an address:

```
# Always present — the safety net
ip -4 route add blackhole default metric 32767 table 200

# Present only while nordlynx is up and addressed
ip -4 route add default dev nordlynx metric 10 table 200
```

When the tunnel drops, the reconciliation loop removes the VPN route. The kernel then falls through to the blackhole, and the device gets nothing. Crucially, the safe state is the *default* state. Nothing has to fire

correctly for the device to stay protected. Protection is what happens when nothing happens. That is the essence of fail-closed design. It is the same reasoning behind deny-by-default network security groups and Conditional Access policies in the cloud world.

One subtler leak path deserves a mention. Suppose another tool flushes the nftables tables while the gateway service is dead. Forwarded traffic would then fall back to the main routing table, which has a perfectly good default route to the ISP. This is why the data plane runs as a supervised reconciliation loop rather than a one-shot script. Every few seconds it verifies the policy rules, the blackhole route, and both nftables tables. It reinstates anything missing. Reconciliation loops are not just for Kubernetes.

nftables: deny by default at the gateway

The forward path only accepts registered devices, and only toward the tunnel:

```
chain forward {
    type filter hook forward priority -10; policy accept;

    iifname "eth0" oifname "nordlynx" \
        ip saddr @vpn_clients counter accept

    iifname "nordlynx" oifname "eth0" \
        ip daddr @vpn_clients ct state established,related counter accept

    iifname "eth0" counter drop
}
```

The last rule matters more than it looks. Someone may manually point an unmanaged LAN device at the gateway. That device gets dropped rather than forwarded. Without this rule, a misconfigured device would happily use the VM as a clear-net router. That is exactly the accidental fallback this project exists to prevent. Masquerading applies only to traffic from the managed set leaving through the tunnel interface, so nothing else can co-opt the NAT rule.

The DNS side channel, and what enforcement really means

Here is the part most VPN gateway scripts get wrong. A managed TV often keeps the LAN router as its DNS server. Its DNS packets then never touch the gateway at all. They travel directly across the subnet to the router and out through the ISP. Your traffic goes through the tunnel while your DNS announces your real location.

Setting the gateway as the DNS server on each managed device is still a manual step. Same-subnet DNS traffic sent directly to the LAN router never reaches the gateway, and no routing trick can intercept it. What the gateway does enforce is everything after that: only registered devices may use the local resolver, and every upstream query dnsmasq generates is forced through the same fail-closed routing table as application traffic. The configuration is manual; the guarantee, once configured, is not.

The mechanism works like this. The gateway runs a local dnsmasq instance under a dedicated system user, `vpn-dns`. Managed devices use the gateway as their DNS server, and dnsmasq forwards to the VPN provider’s resolvers. The elegant bit is how the proxy’s own upstream traffic gets forced through the tunnel: a UID-based policy rule.

```
ip -4 rule add priority 9999 uidrange <vpn-dns-uid>-<vpn-dns-uid> table 200
```

Every packet generated by the `vpn-dns` user routes through the same table as the managed devices. VPN route when the tunnel is up, blackhole when it is not. DNS inherits the exact same fail-closed guarantee as the data path, with zero application-level logic. On top of that, an nftables input chain accepts port 53 only from registered devices and drops everyone else. The resolver cannot quietly become an open LAN service.

A related question that comes up: what about WebRTC leaks? That leak class belongs to on-device VPN clients, where browser STUN traffic can escape a split tunnel. A routed gateway has no split path to escape to. STUN rides the same fail-closed routing table as everything else, so a WebRTC check on a managed

device returns the VPN exit address. It may expose a private RFC1918 LAN address, but it does not reveal the ISP-facing public IP.

The trap that locks you out: the VPN client's own firewall

One operational detail costs people hours. When the NordVPN Linux client connects, its own firewall blocks inbound LAN connections to the host. You connect once, and your SSH session and web panel are gone. The fix is an exact subnet allowlist. But the client refuses to add a private subnet while LAN Discovery is enabled. So you must disable one setting and then add the other. Between the two commands sits a window where your SSH session can freeze mid-transition. That leaves the machine locked out permanently.

The installer solves this with a technique worth stealing. It runs both operations inside a **transient systemd unit** via `systemd-run --wait`. The unit belongs to the machine, not to the SSH session. Even if the connection pauses, the transition completes locally. An ERR trap restores LAN Discovery if the allowlist step fails. It is a two-line change-management problem, and it gets the same treatment a production migration would: make the change atomic from the operator's point of view, and define the rollback before executing.

Least privilege, enforced by systemd

Three services run at three privilege levels. The unit files declare the boundaries; documentation merely describes them.

- **Gateway (root)** — it still runs as root because kernel routing and nftables require elevated privileges, but its capability set and writable filesystem paths are tightly constrained. `CapabilityBoundingSet=CAP_NET_ADMIN CAP_DAC_READ_SEARCH`, `ProtectSystem=strict`, `ProtectHome=yes`, and `MemoryDenyWriteExecute=yes` reduce the available attack surface without pretending that a privileged shell service is risk-free.
- **DNS proxy (vpn-dns)** — an unprivileged system user holding only `CAP_NET_BIND_SERVICE` to bind port 53.
- **Web panel (regular user)** — no capabilities at all. It writes one JSON file and calls the NordVPN daemon through group membership. Even a full compromise of the Flask process cannot modify kernel policy rules or nftables state — only the privileged gateway service reconciles those. What a compromised panel could do is invoke the permitted NordVPN client operations: connect, disconnect, change country. Note what the worst case is. A forced disconnect takes managed devices offline, because fail-closed turns the nastiest thing an attacker can do into an availability problem rather than a privacy one.

Startup ordering is part of the security model too. The gateway service starts `Before=` the DNS proxy and the panel. That closes the brief window where a resolver could be reachable before the nftables protections exist. Ordering bugs are security bugs. systemd just happens to fix them declaratively.

Transactional updates with a health gate

The updater treats an upgrade the way a database treats a transaction. Before touching anything, it snapshots every managed file: application code, all three unit files, the environment file, and the runtime configuration. It then stops the services, disables IP forwarding (fail closed during the change window as well), installs the new versions, and restarts.

Then comes the part that separates "restart and hope" from an actual deployment. The updater polls a health heartbeat. It refuses to declare success until the gateway reports a *protected* state: fail-closed route present, both nftables tables present, policy rule count matching the device count, DNS proxy active, and the version string matching the release. If that state does not arrive within the timeout, an ERR trap restores every snapshot, reloads systemd, restarts the previous version, and exits non-zero. Either the new version proves itself healthy, or the previous one comes back — the gateway is never left half-applied.

The heartbeat itself is a JSON file the gateway writes on every reconciliation cycle. The web panel renders it as a status card: healthy, fail-closed (protected but tunnel down), or degraded. The distinction between *fail-closed* and *degraded* is deliberate. "The VPN is down and your devices are safely offline" and "the

protection itself is broken" are very different situations. A status page that cannot tell them apart is not a status page.

Verifying the claims

A design document is worth little without a way to check it against reality. The repository ships a smoke test that validates the installed gateway end to end. Its optional `--with-failover` mode disconnects the VPN and proves that both routing and DNS actually fail closed before reconnecting. Before running its assertions, the smoke test waits for the connected gateway state to converge, which prevents false route or DNS failures immediately after an installation or update. The CI pipeline holds the code to the same standard: shellcheck and ruff for lint, pytest for the validation logic, `nft -c` against the rendered ruleset, `systemd-analyze verify` against the unit files, and a version-consistency check across the application, the installer, and the heartbeat.

The gateway is designed for a small Ubuntu Server host or VM with systemd, bridged LAN connectivity, a fixed IPv4 address, and the official NordVPN Linux client.

Takeaways

Compress the project into principles, and none of them are about VPNs:

- **Make the safe state the default state.** The blackhole route protects devices precisely because it requires nothing to happen.
- **Enforce what you can, and be honest about what you can't.** The DNS design enforces the resolution path from the gateway outward — and states plainly that the per-device setting remains manual.
- **Reconcile continuously.** Declared state drifts. A loop that repairs drift beats a script that assumes none.
- **Changes are transactions.** Snapshot, apply, verify health, or roll back — whether the system is a cloud landing zone or a homelab VM.
- **Privilege boundaries belong in configuration, not in comments.** systemd sandboxing turns "this service shouldn't be able to..." into "this service is not allowed to."

A fail-closed VPN gateway is a small system, but it rewards the same engineering discipline as a large one.

Source & documentation (MIT): github.com/vdionisopoulos/nordvpn-linux-gateway-panel
Original article: cloudtales.gr/fail-closed-vpn-gateway-linux/
Author: Vassilis Dionysopoulos — cloud architecture notes at cloudtales.gr